

Отчёт по лабораторной работе №2 по ООП
Забелич Павел Алексеевич 451001

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using Lab2.Models;
using Newtonsoft.Json;

namespace Lab2.Serialization;

public sealed class JsonFleetSerializer : IFleetSerializer
{
    private static readonly JsonSerializer Serializer = new()
    {
        Formatting = Formatting.Indented,
        Culture = System.Globalization.CultureInfo.InvariantCulture
    };

    public void Serialize(Stream output, IReadOnlyList<SpaceVessel> fleet)
    {
        var mementos = fleet.SelectToMementos();
        using var writer = new StreamWriter(output, new
            UTF8Encoding(encoderShouldEmitUTF8Identifier: false), bufferSize: 4096, leaveOpen: true);
        using var jsonWriter = new JsonTextWriter(writer) { Formatting = Formatting.Indented };
        Serializer.Serialize(jsonWriter, mementos);
    }

    public IReadOnlyList<SpaceVessel> Deserialize(Stream input)
    {
        using var reader = new StreamReader(input, Encoding.UTF8,
            detectEncodingFromByteOrderMarks: true, bufferSize: 4096, leaveOpen: true);
        using var jsonReader = new JsonTextReader(reader);
        var mementos = Serializer.Deserialize<List<VesselMemento>>(jsonReader)
            ?? throw new FormatException("Пустой или некорректный JSON.");
        var vessels = new List<SpaceVessel>(mementos.Count);
        foreach (var m in mementos)
            vessels.Add(m.ToVessel());
        return vessels;
    }

    internal static class FleetSerializationLinq
    {
        public static List<VesselMemento> SelectToMementos(this IReadOnlyList<SpaceVessel> fleet)
        {
            var list = new List<VesselMemento>(fleet.Count);
            foreach (var v in fleet)
                list.Add(VesselMemento.FromVessel(v));
            return list;
        }
    }
}
```

```
}  
}
```

```
using System;  
using System.Collections.Generic;  
using System.Globalization;  
using System.IO;  
using System.Text;  
using Lab2.Models;
```

```
namespace Lab2.Serialization;
```

```
/// <summary>Собственный текстовый формат: заголовок версии и блоки VESSEL ...  
END.</summary>
```

```
public sealed class TextFleetSerializer : IFleetSerializer
```

```
{
```

```
    private const string Header = "LAB2_FLEET_TXT_V1";
```

```
    private const string Begin = "BEGIN_VESSEL";
```

```
    private const string End = "END_VESSEL";
```

```
    public void Serialize(Stream output, IReadOnlyList<SpaceVessel> fleet)
```

```
    {
```

```
        using var writer = new StreamWriter(output, new  
        UTF8Encoding(encoderShouldEmitUTF8Identifier: false), bufferSize: 1024, leaveOpen: true);
```

```
        writer.WriteLine(Header);
```

```
        foreach (var vessel in fleet)
```

```
        {
```

```
            var m = VesselMemento.FromVessel(vessel);
```

```
            writer.WriteLine(Begin);
```

```
            writer.WriteLine($"TypeName={m.TypeName}");
```

```
            writer.WriteLine($"Uid={m.Uid}");
```

```
            writer.WriteLine($"ModelName={Escape(m.ModelName)}");
```

```
            writer.WriteLine($"Fuel={m.Fuel.ToString(CultureInfo.InvariantCulture)}");
```

```
            writer.WriteLine(m.FirePower.HasValue
```

```
                ? $"FirePower={m.FirePower.Value.ToString(CultureInfo.InvariantCulture)}"
```

```
                : "FirePower=");
```

```
            writer.WriteLine(m.Capacity.HasValue
```

```
                ? $"Capacity={m.Capacity.Value.ToString(CultureInfo.InvariantCulture)}"
```

```
                : "Capacity=");
```

```
            writer.WriteLine(End);
```

```
        }
```

```
    }
```

```
    public IReadOnlyList<SpaceVessel> Deserialize(Stream input)
```

```
    {
```

```
        using var reader = new StreamReader(input, Encoding.UTF8,  
        detectEncodingFromByteOrderMarks: true, bufferSize: 1024, leaveOpen: true);
```

```
        var lines = reader.ReadToEnd().Split(new[] { "\r\n", "\n" }, StringSplitOptions.None);
```

```
        if (lines.Length == 0 || lines[0].Trim() != Header)
```

```
            throw new FormatException($"Ожидался заголовок '{Header}'");
```

```
var list = new List<SpaceVessel>();
for (var i = 1; i < lines.Length; i++)
{
    if (lines[i].Trim() != Begin)
        continue;
```

```
var props = new Dictionary<string, string>(StringComparer.OrdinalIgnoreCase);
i++;
while (i < lines.Length && lines[i].Trim() != End)
{
    var line = lines[i];
    var eq = line.IndexOf('=');
    if (eq > 0)
    {
        var key = line[..eq].Trim();
        var value = line[(eq + 1)..];
        props[key] = Unescape(value);
    }
    i++;
}
```

```
list.Add(ParseVessel(props));
}
```

```
return list;
}
```

```
private static SpaceVessel ParseVessel(Dictionary<string, string> props)
{
    string Req(string k) =>
        props.TryGetValue(k, out var v) ? v : throw new FormatException($"Отсутствует поле '{k}'.");
```

```
var m = new VesselMemento
{
    TypeName = Req("TypeName"),
    Uid = Req("Uid"),
    ModelName = Req("ModelName"),
    Fuel = double.Parse(Req("Fuel"), CultureInfo.InvariantCulture)
};
```

```
if (props.TryGetValue("FirePower", out var fp) && !string.IsNullOrWhiteSpace(fp))
    m.FirePower = int.Parse(fp, CultureInfo.InvariantCulture);
```

```
if (props.TryGetValue("Capacity", out var cap) && !string.IsNullOrWhiteSpace(cap))
    m.Capacity = int.Parse(cap, CultureInfo.InvariantCulture);
```

```
return m.ToVessel();
}
```

```
private static string Escape(string s) =>
s.Replace("\\", "\\\\", StringComparison.Ordinal).Replace("\n", "\\n", StringComparison.Ordinal);
```

```
private static string Unescape(string s) =>
s.Replace("\\\\", "\0", StringComparison.Ordinal)
.Replace("\\n", "\n", StringComparison.Ordinal)
.Replace("\\0", "\0", StringComparison.Ordinal);
}
```

```
using System;
using System.Collections.Generic;
using System.IO;
using Lab2.Models;
using MessagePack;
using MessagePack.Resolvers;
```

```
namespace Lab2.Serialization;
```

```
public sealed class BinaryFleetSerializer : IFleetSerializer
{
    private static readonly MessagePackSerializerOptions Options =
    MessagePackSerializerOptions.Standard.WithResolver(StandardResolver.Instance);
```

```
public void Serialize(Stream output, IReadOnlyList<SpaceVessel> fleet)
{
    var mementos = fleet.SelectToMementos();
    MessagePackSerializer.Serialize(output, mementos, Options);
}
```

```
public IReadOnlyList<SpaceVessel> Deserialize(Stream input)
{
    var mementos = MessagePackSerializer.Deserialize<List<VesselMemento>>(input, Options)
    ?? throw new FormatException("Пустой или повреждённый бинарный файл.");
    var vessels = new List<SpaceVessel>(mementos.Count);
    foreach (var m in mementos)
        vessels.Add(m.ToVessel());
    return vessels;
}
```

```
using System.Collections.Generic;
using System.IO;
using Lab2.Models;
```

```
namespace Lab2.Serialization;
```

```
public interface IFleetSerializer
{
    void Serialize(Stream output, IReadOnlyList<SpaceVessel> fleet);
}
```

```
ReadOnlyList<SpaceVessel> Deserialize(Stream input);  
}
```

```
using System;  
using MessagePack;
```

```
namespace Lab2.Models;
```

```
[MessagePackObject]  
public sealed class VesselMemento  
{  
    [Key(0)] public string TypeName { get; set; } = "";
```

```
    [Key(1)] public string Uid { get; set; } = "";
```

```
    [Key(2)] public string ModelName { get; set; } = "";
```

```
    [Key(3)] public double Fuel { get; set; }
```

```
    [Key(4)] public int? FirePower { get; set; }
```

```
    [Key(5)] public int? Capacity { get; set; }
```

```
    public static VesselMemento FromVessel(SpaceVessel vessel)  
    {  
        var m = new VesselMemento  
        {  
            TypeName = vessel.GetType().Name,  
            Uid = vessel.Uid,  
            ModelName = vessel.ModelName,  
            Fuel = vessel.Fuel  
        };  
    }
```

```
    switch (vessel)  
    {  
        case CombatShip combat:  
            m.FirePower = combat.FirePower;  
            break;  
        case TransportShip transport:  
            m.Capacity = transport.Capacity;  
            break;  
    }
```

```
    return m;  
}
```

```
    public SpaceVessel ToVessel()  
    {  
        SpaceVessel vessel = TypeName switch
```

```
{  
nameof(CargoFreighter) => new CargoFreighter(ModelName),  
nameof(Destroyer) => new Destroyer(ModelName),  
nameof(ScoutFighter) => new ScoutFighter(ModelName),  
_ => throw new FormatException($"Неизвестный тип корабля: '{TypeName}'.")  
};
```

```
vessel.ApplyDeserializedIdentity(Uid, ModelName, Fuel);
```

```
if (vessel is CombatShip combat && FirePower.HasValue)  
combat.FirePower = FirePower.Value;
```

```
if (vessel is TransportShip transport && Capacity.HasValue)  
transport.Capacity = Capacity.Value;
```

```
return vessel;
```

```
}  
}
```